HANDWRITTEN DIGIT RECOGNITION

Building a CNN That Reads Your Handwriting

You draw a digit with your mouse.
The neural network tells you what it sees.
In real time.

THE STORY SO FAR

Last session we built a Perceptron — the simplest neural network possible.

One neuron. 3 inputs. Weights × inputs → sum → sign() → +1 or -1.
It learned to classify university admissions from exam scores.
It learned to paint shapes based on mouse position.

But it could only draw straight lines. Anything curved or enclosed — it failed.

Session 1 ✓

Perceptron

1 neuron, 3 weights
Straight lines only

Session 2 ✓

Shape Painter + Exercise

Applied the perceptron
Saw the limitation

TODAY ←

CNN + Digit Recognition

7 layers, 56,714 weights
Recognizes handwriting

Next

Neuroevolution

Evolve NN weights
with GA from Ch.9

Same core idea: inputs × weights → sum → activation → output. But scaled up massively.

WHAT WE'RE BUILDING TODAY

1. You draw

A digit (0–9) on a 280×280 canvas with your mouse. White ink on black.

2. Shrink it

Your drawing gets resized to 28×28 pixels. That's 784 tiny brightness values.

3. Feed it to the CNN

784 numbers flow through 7 layers. Each layer transforms the data.

4. Get a prediction

The network outputs 10 probabilities. The digit with the highest % wins.

The CNN was trained on 15,000 real handwritten digits from the MNIST dataset.
It learned what digits look like by studying thousands of examples — just like we learned the perceptron.

WHAT IS A CNN?

CNN = Convolutional Neural Network

A neural network specially designed for images.

A regular neural network (like our perceptron) sees 784 individual pixels.
It doesn't know that pixel 15 is NEXT TO pixel 16. It sees a flat list of numbers.

A CNN uses "filters" that slide across the image looking for local patterns:
Edges, curves, corners. It understands that pixels next to each other matter.

How your brain sees a digit:

First you see lines and edges.
Then you see shapes made of edges (curves, corners, loops).
Then you recognize the number.

A CNN works the same way:
Layer 1 finds edges.
Layer 2 finds shapes made of edges.
The final layers recognize the digit.

This is called hierarchical feature learning.

WHAT IS TENSORFLOW?

TensorFlow is Google's open-source machine learning library.
It's the engine that powers Google Search, Google Photos, Google Translate, Gmail spam filters, YouTube recommendations, and Waymo self-driving cars.

TensorFlow.js is the JavaScript version. Same engine, runs in your browser.
No installation. No Python. No server. Just a <script> tag.

Why "Tensor" + "Flow"?

Tensor = a multi-dimensional array of numbers.
(Next slide explains this in detail.)

Flow = data FLOWS through a network of operations, layer by layer.

TensorFlow = numbers flowing through a neural network. That's literally it.

What it gives us:

- Pre-built layer types (conv2d, dense, etc.)
- Automatic backpropagation (training)
- GPU acceleration (100× faster than CPU)
- model.fit() — trains with one function call
- model.predict() — classifies with one call

Without TensorFlow, we'd have to write thousands of lines of calculus by hand.
With it, our entire CNN is ~20 lines.

WHAT IS A TENSOR?

The library is called TensorFlow. So what's a Tensor?

A tensor is just a container for numbers — but it can hold them in different shapes.

0D: A single number

42

A score on a test.
Just one value.

```
tf.scalar(42)
```

1D: A list of numbers

[10, 20, 30, 40]

A row of exam scores.
Or one row of pixels.

```
tf.tensor1d([10,20,30,40
])
```

2D: A grid of numbers

```
[[1,2,3],
 [4,5,6],
 [7,8,9]]
```

A spreadsheet. Or a grayscale image
(rows × cols of brightness).

```
tf.tensor2d([[1,2],[3,4]
])
```

4D: Our image tensor

[1, 28, 28, 1]

1 image, 28 pixels wide,
28 pixels tall, 1 color
channel (grayscale).

This is what our CNN eats.

```
tf.tensor4d(pixels,
[1, 28, 28, 1])
```

Why not use normal JavaScript arrays? Tensors live on the GPU. They're 100× faster for math.
TensorFlow can only do its matrix multiplication magic on tensors, not on regular arrays.

STEP 1: THE ASSEMBLY LINE

```
model = tf.sequential();
```

What is tf.sequential()?

tf = TensorFlow.js (the AI library Google built).
sequential = a model where data flows in one direction only.

Think of a car factory assembly line:
- Raw steel enters the front door
- Station 1 bends it into a frame
- Station 2 welds the body
- Station 3 paints it
- Station 4 installs the engine
- A finished car drives out the back door

Data never goes backward. It never skips a step.

In our case: raw pixels enter → 7 stations transform them → a digit prediction comes out.

This line creates an empty assembly line. Next we add the stations (layers) one by one.

STATION 1: THE FIRST MAGNIFYING GLASS

```
model.add(tf.layers.conv2d({
  inputShape: [28, 28, 1],
  kernelSize: 3,
  filters: 16,
  activation: "relu"
}));
```

`inputShape: [28, 28, 1]`

"I'm receiving a 28×28 pixel image with 1 color channel (grayscale, not RGB)."

`kernelSize: 3`

"My magnifying glass is 3×3 pixels wide."

`filters: 16`

"I have 16 different magnifying glasses, each looking for a different pattern."

The Toilet Paper Tube Analogy

Imagine looking at a giant painting through a toilet paper tube. You can't see the whole picture.
But if you slide the tube across the canvas, you'll spot things:

- A horizontal silver line (maybe the bumper of a car)
- A black curve (maybe a tire)
- A sharp corner (maybe a window edge)

That's what a convolutional filter does. It's a tiny 3×3 window that slides across the entire image.
At each position, it checks: "Does this 3×3 patch look like the edge I'm searching for?"

This layer has 16 filters — 16 different tubes, each trained to spot a different kind of edge.

`Input: 28×28×1 → Output: 26×26×16 (16 feature maps, slightly smaller because the 3×3 can't reach the very edge)`

WHAT IS activation: “relu” ?

Remember our perceptron’s activation function?

Session 1: sign() ⟶ positive = +1, negative = -1

ReLU is the upgrade. It stands for Rectified Linear Unit.

ReLU

The rule:

```
If negative → make it 0
If positive → let it through
```

Why:

Lightning fast. If a neuron is looking for a loop and doesn’t find one, the math goes negative. ReLU zeroes it out — “turns off” that neuron.

`Used in: Hidden layers (most common)`

Sigmoid

The rule:

```
Squash every number into
a range between 0.0 and 1.0
```

Why:

Great for yes/no probability.
But has a problem in deep networks (vanishing gradient — stops learning).

`Used in: Binary classification`

Softmax

The rule:

```
Make all outputs share a pie
that adds up to 100%
```

Why:

Converts raw scores into probabilities. Each output is a % and they all sum to 1.0.

`Used in: Final layer (multi-class)`

STATION 2: THE SHRINK RAY

```
model.add(tf.layers.maxPooling2d({ poolSize: 2, strides: 2 }));
```

What it does:

poolSize: 2 means look at 2×2 blocks.
strides: 2 means move 2 pixels each step.

For each 2×2 block of pixels:
- Find the BRIGHTEST pixel
- Keep it
- Delete the other three

The image shrinks by 50%.
26×26 → 13×13

Why do this?

1. Speed: fewer pixels = faster computation. Without pooling, the next conv layer would have to process 4× more data.

2. Focus: forces the network to care about WHAT features exist, not exactly WHERE they are. A “3” is still a “3” whether it’s shifted 2 pixels to the left.

Analogy: squinting your eyes.
Or reading a summary of a book.
You lose the tiny details,
but you keep the important parts.

26×26×16 → 13×13×16 (half the size, same number of feature maps)

STATIONS 3-4: THE DETECTIVE

```
model.add(tf.layers.conv2d({
  kernelSize: 3,  filters: 32,  activation: "relu"
}));
model.add(tf.layers.maxPooling2d({ poolSize: 2, strides: 2 }));
```

Same pattern, one level deeper

Layer 1 found raw edges: “a horizontal line here, a curve there.”

Layer 3 is the detective that puts those edges together:

- “I see a vertical line AND a horizontal line meeting → that’s a corner!”
- “I see a curve that loops back on itself → that’s part of a 0, 6, or 9!”
- “I see a straight vertical line with a hook at the bottom → could be a 1 or 7!”

32 filters instead of 16 — because there are far more complex shapes (loops, intersections, hooks, T-junctions) than there are basic edges.

Then MaxPooling shrinks it again: 11×11 → 5×5.

13×13×16 → 11×11×32 → 5×5×32 (smaller but RICHER — more meaning per pixel)

STATION 5: THE BRIDGE

```
model.add(tf.layers.flatten());
```

Why do we need this?

Convolutional layers think in 2D grids (rows and columns of pixels).
Dense layers (the classic neural network) only understand 1D lists.
Flatten bridges the gap between them.

It takes the 5×5×32 grid and unrolls it into one long list of 800 numbers.
No math happens. No learning happens. It just changes the shape.

> *Analogy: You have a Sudoku board (a 2D grid). You want to read it to a friend over the phone.*
> *You can't text them a grid. So you read it left-to-right, top-to-bottom: one long sentence of numbers.*

`5×5×32 grid → flat list of 800 numbers (ready for the dense layers)`

STATION 6: THE BRAIN

```
model.add(tf.layers.dense({ units: 64, activation: "relu"
}));
```

units: 64 means 64 neurons.

This IS the classic neural network.

Every single one of those 800 numbers from the Flatten layer connects to ALL 64 neurons. That's 800 × 64 = 51,200 connections. Each connection has its own weight. That's 51,200 weights to learn.

This is the same idea as our perceptron from Session 1:
inputs × weights ⟶ sum ⟶ activation function ⟶ output

But instead of 1 neuron with 3 inputs, we have 64 neurons each with 800 inputs. Same formula, much bigger.

activation: "relu" — same ReLU as before.
Negative results get zeroed out.
Positive results pass through.

These 64 neurons look at the shape features from the conv layers and try to figure out which digit they represent.

800 numbers ⟶ 64 numbers — 64 distilled features (the network's "opinion" about the shapes it found)

STATION 7: THE DECISION ROOM

```
model.add(tf.layers.dense({ units: 10, activation: "softmax"
}));
```

units: 10 = one neuron per digit

Neuron 0: “How much does this look like a 0?”
Neuron 1: “How much does this look like a 1?”
Neuron 2: “How much does this look like a 2?”
...
Neuron 9: “How much does this look like a 9?”

Each neuron outputs a raw score.
But raw scores are meaningless numbers.

Softmax: Sharing the Pie

Imagine 10 neurons shouting their scores:

Neuron #3 shouts “I score 45!”
Neuron #8 shouts “I score 12!”
Neuron #0 shouts “I score 3!”

Softmax forces them to share a pie:

3 → 80.2% (winner!)
8 → 10.1%
0 → 2.3%
... (all add up to exactly 100%)

Now the output IS a probability.
The highest percentage is our answer.

64 features → 10 probabilities that sum to 1.0 → digit with the highest probability is the prediction

HIRING THE TEACHER

```
model.compile({
  optimizer: "adam",
  loss: "categoricalCrossentropy",
  metrics: ["accuracy"]
});
```

Building a brain doesn't mean it knows how to learn.
compile() sets up the learning system.

loss: categoricalCrossentropy
The grading rubric.
Calculates exactly how wrong the AI's guess was compared to the actual answer key.

optimizer: adam
The teacher.
When the loss says "you were 80% wrong," Adam reaches inside and tweaks the 56,714 weights.

Same idea as our perceptron: error = desired – guess → adjust weights. Adam just does it smarter.

GOING TO SCHOOL: model.fit()

```
await model.fit(
  mnistData.trainImages,
  mnistData.trainLabels,
  {
    epochs: 10,
    batchSize: 128,
    validationData: [
      mnistData.testImages,
      mnistData.testLabels
    ],
    callbacks: {
      onEpochEnd: (e, logs) => {
        epoch = e + 1;
        accuracy = logs.val_acc;
      }
    }
  }
);
```

`await`

Pauses the code until training is done.
Without this, the code would continue before training finishes.

`epochs: 10`

Read the textbook 10 times.
Each epoch = one complete pass through all 15,000 training images.

`batchSize: 128`

Don't swallow all 15,000 images at once.
Show 128 flashcards at a time.
Guess → grade → adjust → next stack.

`validationData`

1,000 images the AI NEVER trains on.
A pop quiz to prevent cheating.
If it memorizes, the quiz catches it.

`onEpochEnd`

TensorFlow taps us on the shoulder:
"I just finished a full pass!"
We save the accuracy for our UI.

This one function adjusts 56,714 weights by showing the network 15,000 images × 10 times = 150,000 learning steps.

THE MOMENT OF TRUTH: classifyDrawing()

```
// Step 1: Get pixels from canvas
let ctx = gfx.elt.getContext("2d");
let imgData = ctx.getImageData(0,0,28,28);
let pixels = new Float32Array(784);
for (let i = 0; i < 784; i++) {
  pixels[i] = imgData.data[i*4] / 255.0;
}
```

```
// Step 2: AI prediction
tf.tidy(() => {
  let tensor = tf.tensor4d(
    pixels, [1, 28, 28, 1]
  );
  let output = model.predict(tensor);
  let probs = output.dataSync();
  // find the winner...
});
```

Standard JavaScript — no AI here yet.

Grab the pixel data from the raw HTML canvas underneath p5's graphics.

getImageData returns RGBA values:
[R, G, B, A, R, G, B, A, ...]
We grab every 4th value (red channel) because it's grayscale (R=G=B).

/ 255.0 normalizes to 0.0–1.0.
The network was trained on 0–1 values.

tf.tidy()
CRITICAL. We predict 60×/second.
Without this garbage collector,
the browser crashes in 5 seconds.

tf.tensor4d(pixels, [1,28,28,1])
Wrap 784 JS numbers into a Tensor.
[1, 28, 28, 1] means: 1 image,
28 wide, 28 tall, 1 color channel.

model.predict(tensor)
THE PAYOFF. In ~1 millisecond,
784 pixels fly through all 7 layers.
Result: 10 probabilities.

output.dataSync()
The result is on the GPU.
dataSync() pulls it back into
regular JavaScript we can use.

FINDING THE WINNER

```
prediction = 0;
confidences = [];

for (let i = 0; i < 10; i++) {
  confidences.push(probs[i]);
  if (probs[i] > probs[prediction]) {
    prediction = i;
  }
}
```

What the AI actually outputs:

The AI doesn't say "It's a 3." It says:

"0.2% chance it's a 0, 0.1% chance it's a 1, 1.5% chance it's a 2,
96.1% chance it's a 3, 0.8% chance it's a 4, 0.1% chance it's a 5,
0.3% chance it's a 6, 0.1% chance it's a 7, 0.7% chance it's a 8, 0.1% chance it's a 9"

We loop through all 10 slots. Save every score for the green bars.
Whichever index has the highest score = our prediction.

THE FULL PICTURE: 56,714 PARAMETERS

Layer	Shape transformation	Weights	Job
Conv2D (16)	28×28×1 → 26×26×16	160	Find edges
MaxPool	26×26×16 → 13×13×16	0	Shrink
Conv2D (32)	13×13×16 → 11×11×32	4,640	Find shapes
MaxPool	11×11×32 → 5×5×32	0	Shrink
Flatten	5×5×32 → 800	0	Reshape
Dense (64)	800 → 64	51,264	Think
Dense (10)	64 → 10	650	Decide

Total: 56,714 learnable parameters | 7 layers | 10 classes | ~97% accuracy | Runs in your browser

LET'S BUILD IT

01	**Open the starter file**	Drawing canvas + MNIST loader + UI — already written for you
02	**Build the CNN together**	7 model.add() calls. We just walked through every single one.
03	**Write model.compile()**	Optimizer + loss function. The grading system.
04	**Write trainModel()**	model.fit() with epochs, batches, callbacks.
05	**Write classifyDrawing()**	Extract pixels → tensor → predict → find the winner.
06	**Press ENTER and watch**	Train. Draw digits. The wow moment.

From 784 raw pixels to a digit prediction — in real time — built by you.